

■ Hoja de Referencia Rápida: Spring Security & JWT (RBAC)

■ 1. Anotaciones de Seguridad y Autorización

Anotación	Propósito	Ejemplo
<code>@EnableWebSecurity</code>	Habilita el soporte de seguridad Web MVC.	<code>@EnableWebSecurity</code>
<code>@EnableMethodSecurity</code>	Activa la seguridad a nivel de método.	<code>@EnableMethodSecurity</code>
<code>@PreAuthorize</code>	Evalúa expresión SpEL antes de ejecutar.	<code>@PreAuthorize("...")</code>
<code>@PostAuthorize</code>	Evalúa expresión SpEL después de ejecutar.	<code>@PostAuthorize("...")</code>
<code>@AuthenticationPrincipal</code>	Inyecta el objeto <code> UserDetails </code> autenticado.	<code>@AuthenticationPrincipal</code>
<code>@Secured</code>	Verificación basada en nombres de roles.	<code>@Secured("ROLE_ADMIN")</code>

■■ 2. Configuración de `SecurityFilterChain` (Spring Boot 3.x)

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .cors(Customizer.withDefaults())
            .sessionManagement(s ->
                s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(
                jwtFilter,
                UsernamePasswordAuthenticationFilter.class
            );

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}

```

■ 3. Expresiones SpEL Comunes en @PreAuthorize

```
@Service
public class DocumentoService {

    @PreAuthorize("hasRole('ADMIN')")
    public void eliminarDocumento(Long id) {}

    @PreAuthorize("hasAnyRole('ADMIN', 'SUPERVISOR')")
    public List<Documento> listarAuditoria() {
        return List.of();
    }

    @PreAuthorize("hasAuthority('SCOPE_write') or " +
        "#clienteId == authentication.principal.id")
    public void actualizarCliente(Long clienteId, ClienteDTO dto) {}

    @PreAuthorize("isAuthenticated()")
    public UsuarioDTO obtenerPerfil() {
        return null;
    }
}
```

■ 4. Filtro Autenticador JWT (JwtAuthenticationFilter)

```
@Component
public class JwtAuthenticationFilter
    extends OncePerRequestFilter {

    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtTokenProvider provider,
        UserDetailsService details) {
        this.tokenProvider = provider;
        this.userDetailsService = details;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
```

```

        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {
    String token = obtenerTokenHeader(request);

    if (StringUtils.hasText(token) &&
        tokenProvider.validarToken(token)) {

        String user = tokenProvider.obtenerUsername(token);
        UserDetails details =
            userDetailsService.loadUserByUsername(user);

        UsernamePasswordAuthenticationToken auth =
            new UsernamePasswordAuthenticationToken(
                details, null, details.getAuthorities()
            );

        auth.setDetails(
            new WebAuthenticationDetailsSource()
                .buildDetails(request)
        );

        SecurityContextHolder.getContext()
            .setAuthentication(auth);
    }

    filterChain.doFilter(request, response);
}

private String obtenerTokenHeader(HttpServletRequest request) {
    String bearer = request.getHeader("Authorization");
    if (StringUtils.hasText(bearer) &&
        bearer.startsWith("Bearer ")) {
        return bearer.substring(7);
    }
    return null;
}
}

```

■ 5. Convención de Roles vs Autoridades

- **Rol (hasRole('ADMIN')):** Spring busca internamente la autoridad con el prefijo ROLE_ADMIN.

- **Autoridad (`hasAuthority('READ_PRIVILEGE')`):** Evalúa la cadena exacta almacenada sin agregar prefijos.